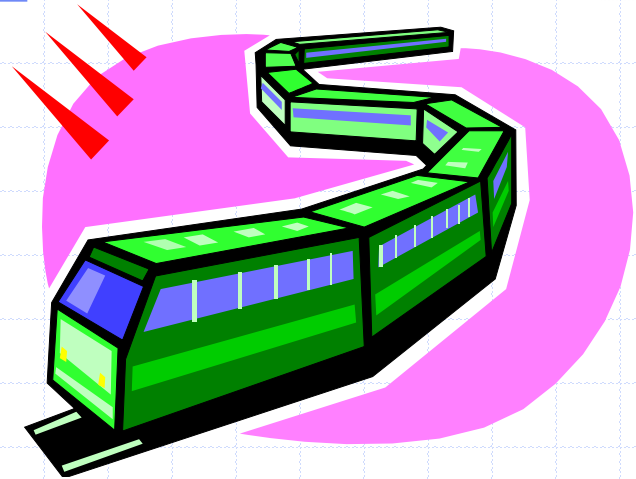


Arrays and Linked Lists



Pointers

- ◆ Pointer holds the value of a memory address
- ◆ The type T^* denotes a pointer to a variable of type T
 - E.g., int^* , char^*
- ◆ The 'address-of' operator, '&', returns the address of a variable
- ◆ Dereferencing
 - Accessing the object addressed by a pointer
 - Done by $*$ operator

Pointers

```
char ch = 'Q';  
char* p = &ch; // p holds the address of ch  
cout << *p;    // outputs the character 'Q'  
ch = 'Z';      // ch now holds 'Z'  
cout << *p;    // outputs the character 'Z'
```

- ◆ Null pointer points to nothing – from c++11, it is denoted using **nullptr**
- ◆ Void type pointer can point to a variable of any type
- ◆ Cannot declare a void type variable

Arrays

- ◆ A collection of elements of the **same type**
- ◆ Index references an element of the array
- ◆ Index is a number from 0 to N-1
- ◆ E.g.,

```
double f[3]; // array of 3 doubles: f[0], f[1], f[2]
double* p[10]; // array of 10 double pointers: p[0], ... , p[9]
f[2] = 25.3;
p[4] = &f[2]; // p[4] points to f[2]
cout << *p[4]; // outputs "25.3"
```

Arrays

◆ Two-dimensional array

- An “array of arrays”
- E.g., `int A[15][30]`

◆ Initializing

```
int    a[4] = {10, 11, 12, 13};    // declares and initializes a[4]
bool   b[2] = {false, true};       // declares and initialize b[2]
char   c[] = {'c', 'a', 't'};      // declares and initialize c[3]
                                     // compiler figures the size of c[]
```

Pointers and Arrays

◆ The name of an array can be used as a pointer to the array's initial element and vice versa

◆ E.g.,

```
char c[] = {'c', 'a', 't'};
char *p = c;           // p point to c[0]
char *q = &c[0];        // q also points to c[0]
cout << c[2] << p[2] << q[2] // outputs "ttt"
```

ADTs for Collections

◆ Collection ADT

- A way to hold multiple objects
must offer.
 - (a) access objects in the container
 - (b) storage
 - (c) iterate / traversal

Collection ADTs

Array : offers access / update by index

List : offers sequential ordering
and insert / delete

Set : ensures objects are unique
(and no ordering guarantees)

Few more ADT Operators

◆ Iterator (traversal) operators

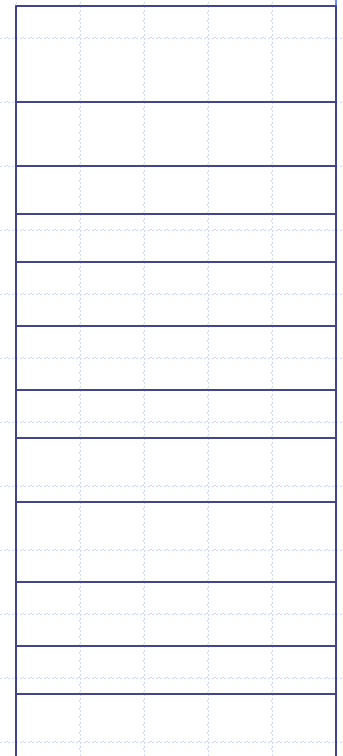
- Applicable when we are working with collection ADTs (will become clearer as we go along)

◆ destructor operator

- It can also be seen as a transformer, but it helps you to “clean up” the ADT when it is no longer required

Arrays

- Memory allocated in contiguous locations
- Must declare size of array
- $A[i]$ can be computed efficiently (1 operation)



Arrays: pluses and minuses

- + Fast element access.
 - Impossible to resize.
-
- Many applications require resizing!
 - Required size not always immediately available.

Arrays: applications

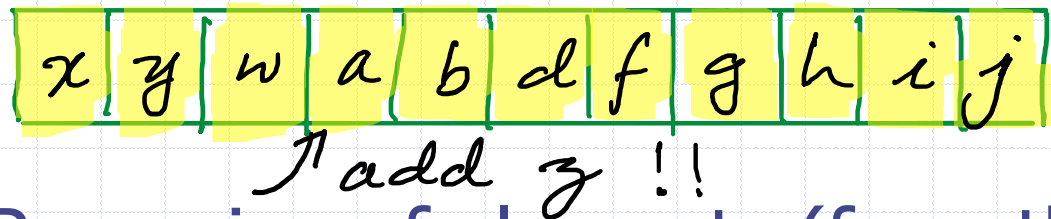
- Very versatile because of simplicity
- Example: polynomial representation

Arrays: applications

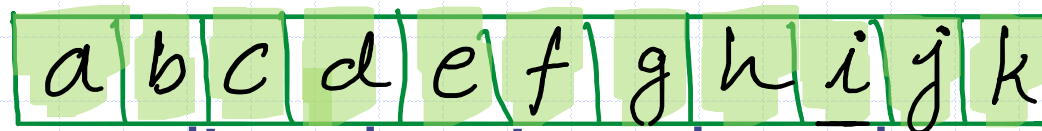
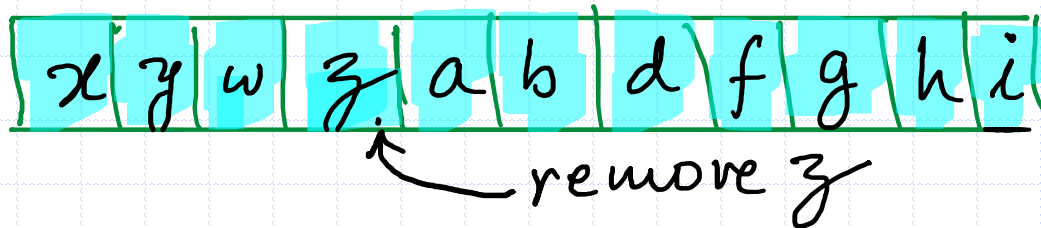
- Matrices (2D arrays) : `int[2][5]`
- Stored linearly
- Not suitable for sparse matrices

Small inconveniences

- ◆ Adding of elements (in the middle)



- ◆ Removing of elements (from the middle)

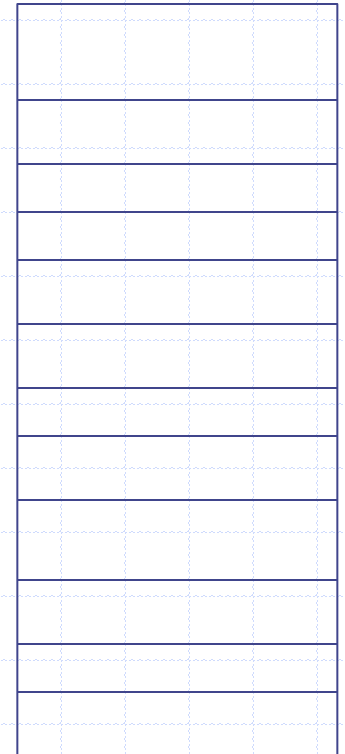


✓ want to
add l

- ◆ Expanding the size when there is a need

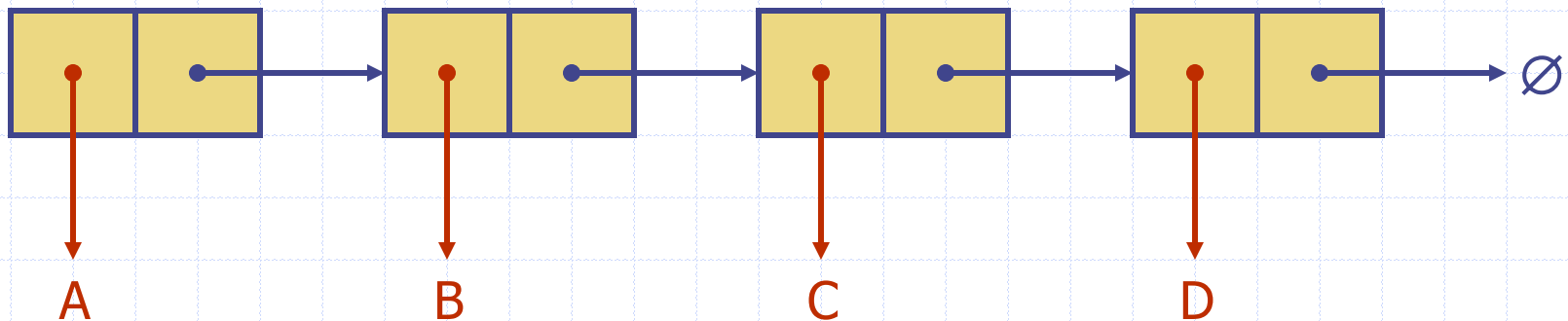
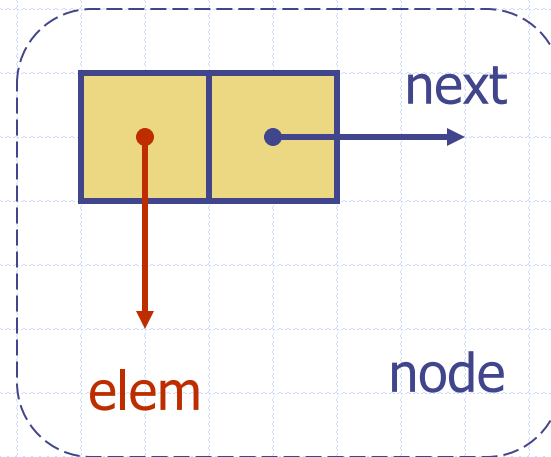
Linked List

- Data is not stored in contiguous locations
- Computation of $A[i]$ expensive
- But easy to change size



Singly Linked Lists

- ◆ A singly linked list is a concrete data structure consisting of a sequence of nodes
- ◆ Each node stores
 - element
 - link to the next node



Recursive Node Class

```
template <typename T>
```

```
class Node {
```

```
private:
```

```
    T element;
```

```
    Node<T>* next;
```

```
public:
```

```
    Node() : element(),  
            next(nullptr) {}
```

```
    Node(const T& e, Node<T>* n  
          = nullptr)  
        : element(e), next(n) {}
```

```
    Node<T>* getNext() const {  
        return next;  
    }
```

```
    const T& getElement()  
    const {  
        return element;  
    }
```

```
    void setElement(const T&  
                    newElem) {  
        element = newElem;  
    }
```

```
    void setNext(Node<T>*  
                 newNext) {  
        next = newNext;  
    }  
};
```

Reference vs Pointer

Feature	Reference (&)	Pointer (*)
Can be null?	✗ No	✓ Yes (<code>nullptr</code>)
Can be reseated?	✗ No	✓ Yes
Stores an address?	Internally yes, but behaves as an alias	✓ Explicitly
Needs dereferencing (<code>*p</code>)?	✗ No	✓ Yes
Best use	Function parameters, aliases	Linked structures, optional relationships, dynamic memory

Understanding 'const'

Why use `const` ?

- ☒ Prevent accidental modification
- ☒ Avoid unnecessary copies
- ☒ Makes interfaces clearer
- ☒ Enables compiler optimizations

◆ Constant variables

- `const double PI = 3.14159;`

◆ Constant reference (avoids copying, method can't modify)

- `void method(const std::string& s)`

◆ Constant member functions (won't modify object state)

- `int size() const`

◆ Returning a constant reference (No copy of the top element; caller cannot modify it)

- `const T& getElement() const`

Singly linked list

```
template <typename T>
class SLinkedList {
protected:
    Node<T>* head;    // Pointer to the head node

public:
    // Default constructor: creates an empty list
    SLinkedList() : head(nullptr) {}

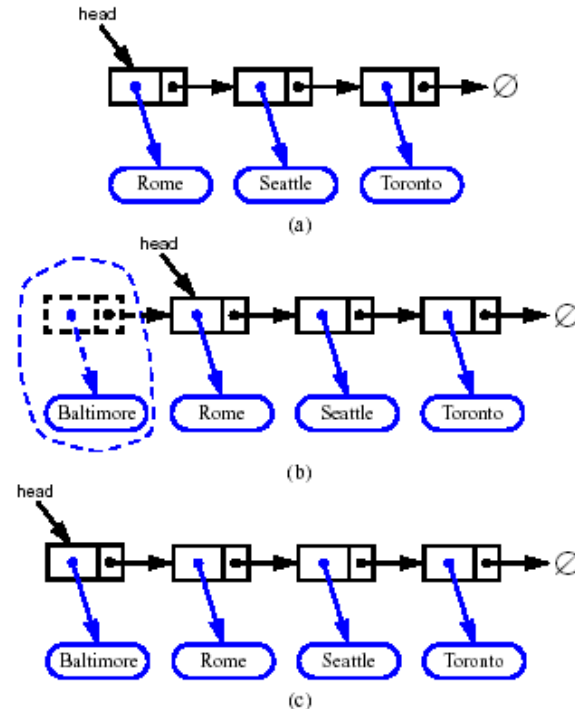
    // Update and search methods would go here...
};
```

Private, Protected, Public

Access Specifier	Same Class	Derived Classes	Other Classes
private	✓	✗	✗
protected	✓	✓	✗
public	✓	✓	✓

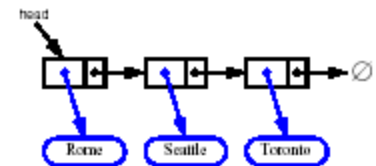
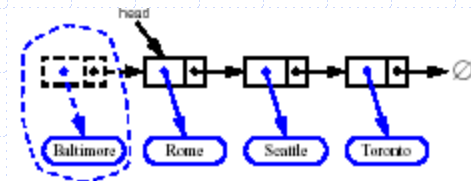
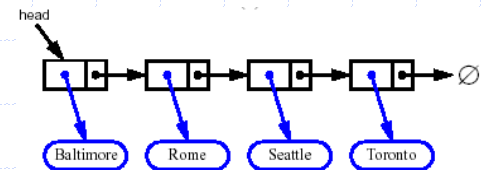
Inserting at the Head

1. Allocate a new node
2. Insert new element
3. Make new node point to old head
4. Update head to point to new node



Removing at the Head

1. Update head to point to next node in the list
2. Delete the former first node



Singly linked list with 'tail' sentinel

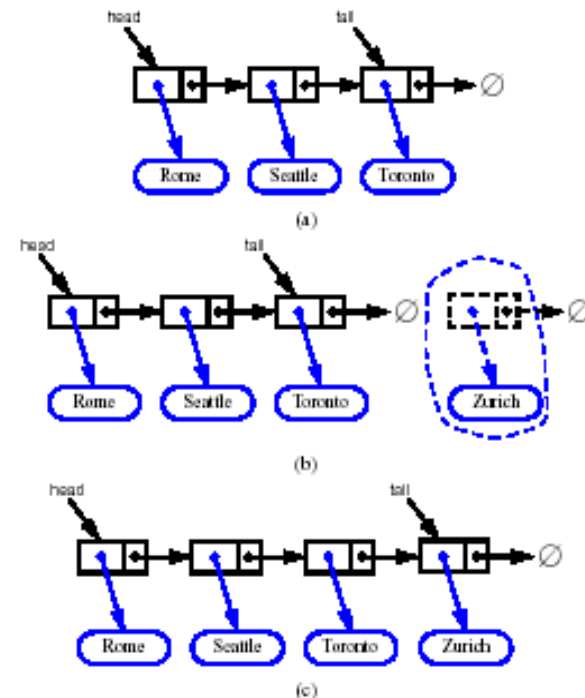
```
template <typename T>
class SLinkedListWithTail : public SLinkedList<T> {
protected:
    Node<T>* tail;

public:
    SLinkedListWithTail()
        : SLinkedList<T>(), tail(nullptr) {}
};
```

Inheritance
Superclass
Derived class

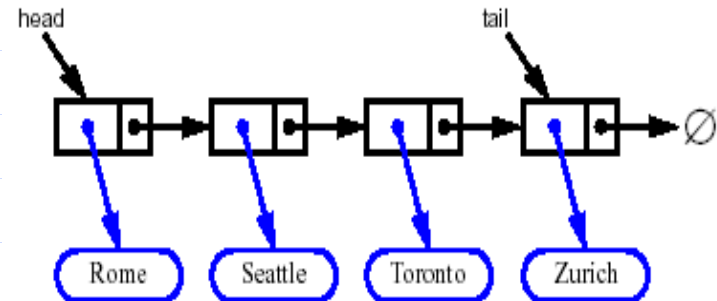
Inserting at the Tail

1. Allocate a new node
2. Insert new element
3. Have new node point to null
4. Have old last node point to new node
5. Update tail to point to new node



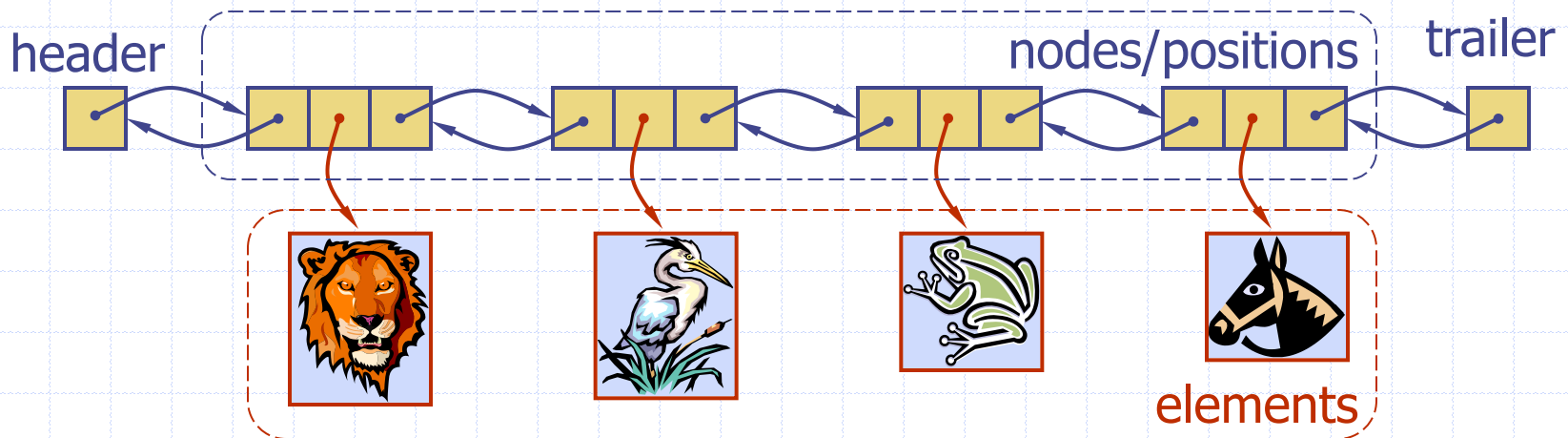
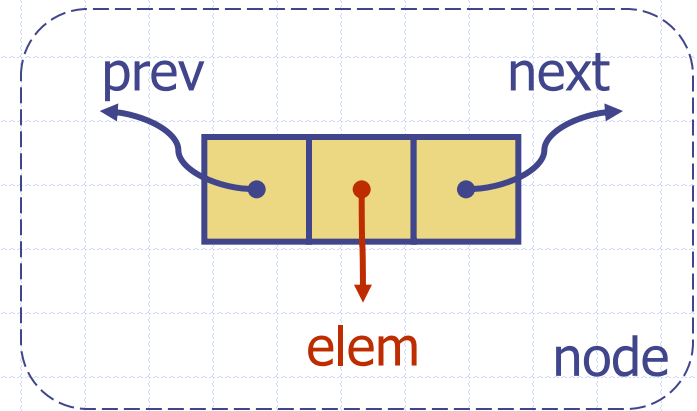
Removing at the Tail

- ◆ Removing at the tail of a singly linked list cannot be efficient!
- ◆ There is no constant-time way to update the tail to point to the previous node



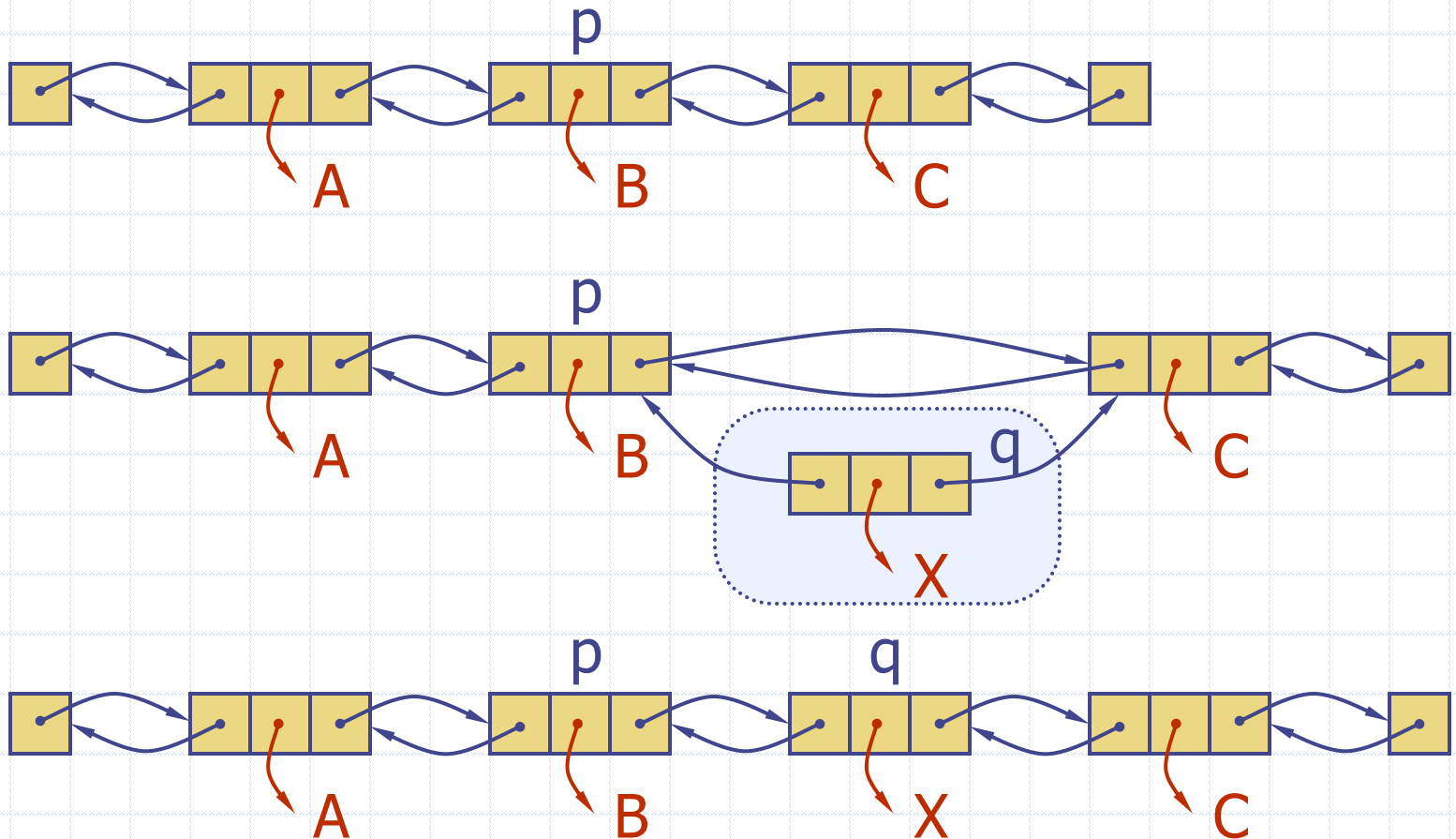
Doubly Linked List

- ◆ A doubly linked list is often more convenient!
- ◆ Nodes store:
 - element
 - link to the previous node
 - link to the next node
- ◆ Special trailer and header nodes



Insertion

- ◆ We visualize operation **insertAfter**(p, X), which returns position q



Insertion Algorithm

Algorithm insertAfter(p, e):

Create a new node v

$v.setElement(e)$

$v.setPrev(p)$ {link v to its predecessor}

$v.setNext(p.getNext())$ {link v to its successor}

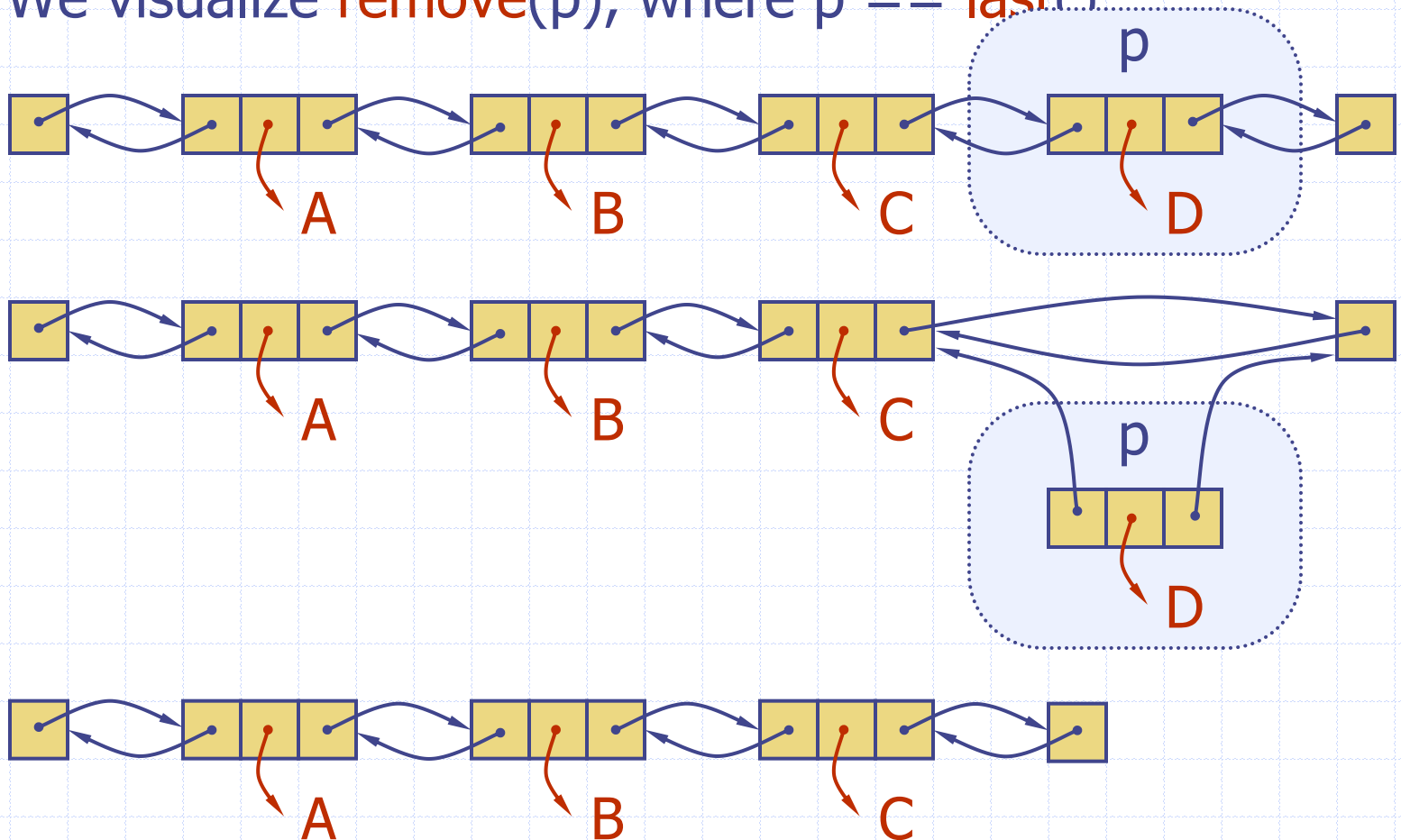
$(p.getNext()).setPrev(v)$ {link p 's old successor to v }

$p.setNext(v)$ {link p to its new successor, v }

return v {the position for the element e }

Deletion

◆ We visualize `remove(p)`, where $p == \text{last}()$



Deletion Algorithm

Algorithm remove(p):

$t = p.\text{element}$ {a temporary variable to hold the return value}

$(p.\text{getPrev}()).\text{setNext}(p.\text{getNext}())$ {linking out p }

$(p.\text{getNext}()).\text{setPrev}(p.\text{getPrev}())$

$p.\text{setPrev}(\text{null})$ {invalidating the position p }

$p.\text{setNext}(\text{null})$

return t

Worst-case running time

◆ In a doubly linked list

- + insertion at head or tail is in $O(1)$
- + deletion at either end is on $O(1)$
- element access is still in $O(n)$